ECA304 PROGRAMAÇÃO ORIENTADA A OBJETOS

Prof. Alexandre H. Moreira
Prof. Anderson H. Moreira

Aula 00: Apresentação da Disciplina

Programa da Disciplina

- Introdução ao Python:
  - Tipos e Operações
  - Controle de Fluxo
  - Estruturas Repetitivas
  - Funções
  - Módulos e Pacotes
- Programação Orientada a Objetos em Python
- Desenvolvimento de Interfaces Gráficas em Python
- Projetos e Aplicações em Python

Avaliação da Disciplina

- Disciplina avaliada somente por trabalhos:
  - Projetos e Atividades em Sala de Aula
  - Projeto extraclasse
- Quatro notas bimestrais (Média dos trabalhos):
  - T1 = 1.0
  - T2 = 1.0
  - T3 = 1.0
  - T4 = 2.0

Python

Sintaxe e Introdução aos tipos básicos de dados

Qual o problema?


```
a = 1
b = 2
c = a + b
print(c)
```

- “;”
- Declaração de variáveis
- Não deveria ser `printf("%i",c)`?

TIPOS BÁSICOS DE DADOS

- Os seguintes tipos são definidos como Python's Core Data Types:

Tipo do Objeto	Exemplos / Criação
Números	1234, 3.1415, 3+4j, 0b111, Decimal(), Fraction()
Strings	spam', "Bob's", b'a\x01c', u'sp\xc4m'
Listas	[1, [2, 'três'], 4.5], list(range(10))
Dicionários	{'nome': 'Anderson', 'profissão': 'Engenheiro'}, dict(horas=10)
Tuplas	(1, 'spam', 4, 'U'), tuple('spam')
Arquivos	open('eggs.txt'), open(r'C:\ham.bin', 'wb')
Conjuntos	set('abc'), {'a', 'b', 'c'}
Outros tipos básicos	Booleanos, tipos, None

IMUTABILIDADE

- Os seguintes tipos são imutáveis:
  - Números
  - Strings
  - Tuplas

```
a = "teste"
a[0] = "T"

T = (1,2,3,3,5)
T[3] = 3
```

Exemplos – Números

- Podemos ter os seguintes tipos para números:
  - Integers: 1, 20, 1987
  - Float: 1.5, 3.14, 1.71
  - Complex: 2+3j, 5.1j, -2.1-3j
  - Decimals: Decimal('1.25')
  - Rationals: Fraction('2.25')
  - Sets: {1,2,3,4,5}

Exemplos – Strings

- Assim como visto em C, as stings são compostas por um conjunto de caracteres.

```
>>> s = 'engeNharia'
>>> print(s.lower())
 engenharia
>>> print(s.upper())
 ENGENHARIA
>>> print(s.capitalize())
 Engenharia
>>> print(s[::-1])
 airahNegne
```

Exemplos – Listas

- As listas em Python são as sequências mais genéricas proporcionadas pela linguagem. As listas são indexadas, podem possuir objetos de diferentes tipos e não possuem tamanho fixo.

```
>>> L = [123, 'teste', 1.23]
>>> L[0]
 123
>>> L[:-1]
 [123, 'teste']
>>> L + [4, 5, 6]
 [123, 'teste', 1.23, 4, 5, 6]
>>> L * 2
 [123, 'teste', 1.23, 123, 'teste', 1.23]
```

Exemplos – Dicionários

- Os dicionários em Python não são apenas sequências, mas são conhecidas como mapeamentos, que são coleções de objetos organizados por chave ao invés de índices.

```
>>> D = {'Nome': 'Peter Parker', 'Herói': 'Homem-Aranha', 'Vingadores': False}
>>> D['Nome']
 'Peter Parker'
>>> D['Vingadores'] = True
>>> print(D)
 {'Herói': 'Homem-Aranha', 'Vingadores': True, 'Nome': 'Peter Parker'}
>>> D['Status'] = 'Muito Feliz'
>>> print(D)
 {'Status': 'Muito Feliz', 'Herói': 'Homem-Aranha', 'Vingadores': True, 'Nome':
'Peter Parker'}
>>> print('Home-Aranha diz: "Valeu Sony!"')
 'Home-Aranha diz: "Valeu Sony!"
```

EXEMPLOS – TUPLAS

- As Tuplas são como as listas. Entretanto, após sua criação os seus valores não podem ser alterados.

```
>>> T = (1, 2, 7, 5, 3, 9, 2, 10)
>>> T[0]
 1
>>> T[:-1]
 (1, 2, 7, 5, 3, 9, 2)
>>> T + (4, 5, 6)
 (1, 2, 7, 5, 3, 9, 2, 10, 4, 5, 6)
>>> T[2] = 10
 Traceback (most recent call last):
  File "<pyshell#46>", line 1, in <module>
    T[2] = 10
 TypeError: 'tuple' object does not support item assignment
```

PALAVRAS RESERVADAS

- As seguintes palavras são reservadas:

False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

ESTRUTURA DO PROGRAMA

- **Indentação:** Os blocos de código são delimitados por meio de indentação:

```
if True:
    print("True")
else:
    print("False")
```

- Cuidado com a indentação:

```
if True:
    print("True")
  print("Errado")
else:
    print("False")
```

ESTRUTURA DO PROGRAMA

- **Comentários:** Comentários podem ser feitos utilizando '#' ou como se fosse uma entrada de uma string, porém, recomenda-se utilizar o '#'

```
#Este é um comentário em uma única linha

"""Este é um
   comentário em
   múltiplas linhas"""

'''Este também é um comentário
   em múltiplas linhas'''
```

ESTRUTURA DO PROGRAMA

- **Entrada de dados:**

```
x = input("Digite uma string: ") #Se não especificado o retorno da função input é uma String

x = int(input("Digite um número inteiro: ")) #Entrada de um inteiro

x = float(input("Digite um número real: ")) #Entrada de um número real

x = complex(input("Digite um número complexo: ")) #Entrada de um número complexo
```

ESTRUTURA DO PROGRAMA

- **Saída de dados via monitor:**

```
x = 1

print("O valor é",x)

print("O valor é %i"%x)

print("""O valor
é %i"""%x)

print("O"+" valor"+" é "+str(x))

print("O valor é {0}".format(x))
```

ESTRUTURA DO PROGRAMA

- **Operadores aritméticos:**

+	Adição
-	Subtração
*	Multiplicação
/	Divisão real
%	Resto da divisão inteira
**	Expoente
//	Divisão inteira

ESTRUTURA DO PROGRAMA

- **Operadores de comparação:**

==	Igual
!=	Diferente
>	Maior
<	Menor
>=	Maior ou igual
<=	Menor ou igual

- **Operadores de atribuição:**

=	Atribuição simples
+=	Adiciona e atribui
-=	Subtrai e atribui
*=	Multiplica e atribui
/=	Divide e atribui
%=	Atribui o resto da divisão
**=	Eleva e atribui
//=	Atribui a parte inteira da divisão

ESTRUTURA DO PROGRAMA

- **Operadores binários:**

&	AND
|	OR
^	XOR
~	NOT
<<	Desloca para esquerda
>>	Desloca para a direita

ESTRUTURA DO PROGRAMA

- **Operadores lógicos:**

not	NOT
and	AND
or	OR

- Operadores lógicos possuem ordem na execução, similar aos operadores aritméticos:
  - **not** é o primeiro a ser avaliado;
  - **and** é o segundo a ser avaliado;
  - **or** é o último a ser avaliado.

ESTRUTURA DO PROGRAMA

- **Operadores de identidade:**

```
>>> a = 30
>>> b = 30
>>> print(a is b)
  True
>>> b = 20
>>> print(a is b)
  False
```